

RK01: Bounded Responses. Bounded Domains.

From a flat real-time executive to bounded writable-authority domains

Antonio Giacomelli

Version 0.1 • September 2026

Abstract

RK01 takes the fixed-priority, statically bounded real-time model of RK0 and adds an explicit user/kernel boundary and MPU-enforced writable-memory domains. It remains a single firmware image built in one final link. Tasks remain the independently scheduled entities. A domain is neither a process nor a loadable module: it is a frozen set of writable authorities shared by one or more tasks, while each active task retains a private stack aperture.

The boundary changes context switching, object identity, ownership, communication, fault handling, linker layout and service entry. Task-facing objects become opaque generation-checked handles; ordinary calls cross SVC with caller-derived authority and pointer validation; blocking operations use explicit continuations; and communication follows the installed writable authority. This paper describes those mechanisms and their rationale, uses the privileged UART Console task as an end-to-end example, separates hardware constraints from deliberate real-time policy, and interprets the available preemption benchmark without presenting whole-chain throughput as isolated context-switch latency.

Keywords: real-time kernel; memory protection unit; memory domains; system calls; inter-task communication; fault containment; microkernel composition.

Contents

1	Purpose and design position	1
1.1	Threat model	2
2	System model and terminology	2
3	Construction of the memory topology	3
3.1	One image and one final link	3
3.2	Domain source bundles	3
3.3	Sequential build and BOOT process	4
3.4	Why a prebuilt kernel object is not the domain model	5
4	MPU views and context-switch classes	5
4.1	The effective task view	5
4.2	Immutable domain maps	6
5	Object identity, scope and lifecycle	6
5.1	Why a pointer is no longer a sufficient handle	6
5.2	Bounded lifecycle	7
5.3	Scope is separate from possession	7
5.4	Fault-aware cleanup	7
6	The system-call boundary	8
6.1	Two entry paths behind one public API	8
6.2	Immediate and blocking calls	9
6.3	Timing consequence	9
7	Communication follows memory authority	10
7.1	Priority-aware service calls	10
8	UART Console as a complete service example	11
8.1	Topology	11
8.2	Transmit path	11
8.3	What the example proves	12
9	Fault containment and application recovery	12
9.1	What RK01 does on an MPU violation	12
9.2	Containment is not recovery	13
9.3	Example: failover rather than blind restart	14
10	Building an operating system around RK01	14
10.1	Three ways to compose a system around RK01	15
10.1.1	A specialised application with no separate OS layer	15
10.1.2	An integrated user-side system layer	15
10.1.3	A service-oriented system	16

11 Why the boundaries encourage a microkernel-like composition	17
11.1 Placement is a system-design decision	17
11.2 The UART service as a composition example	18
11.3 The real-time cost remains visible	19
11.4 Containment is supplied by the kernel; recovery belongs to the OS	19
11.5 Architectural position	20
12 Real-time properties and their price	20
12.1 What remains bounded	20
12.2 Response-time classes	20
12.3 Preemption-chain measurement	21
12.4 Hardware constraints versus real-time policy	22
13 Design consequences and non-goals	23
13.1 The trusted computing base is reduced, not minimal	23
13.2 Shared code is intentional	23
13.3 Domain boundaries do not replace synchronisation	23
13.4 Static topology is both feature and restriction	23
13.5 Containment needs product policy	23
13.6 “Microkernel-like” is an architectural choice	24
14 Conclusion	24

1 Purpose and design position

RK0 is a flat real-time executive [1]. Application and kernel code share one privileged address space; services are ordinary function calls; application pointers and kernel-object pointers can be passed directly. This gives RK0 unusually low indirection and makes the path from a call site to a scheduling decision easy to inspect.

RK01 does not claim that this model was mistaken. It answers a different requirement: contain a defective ordinary task before it can overwrite kernel RAM, another component's persistent state or a privileged service stack.

The result is not simply RK0 with an MPU driver. A real protection boundary must be reflected consistently in:

- memory placement and startup;
- privilege transitions and system calls;
- task and object identity;
- ownership and visibility rules;
- inter-task communication;
- context-switch work;
- fault cleanup and application recovery;
- response-time analysis.

The scheduler's identity remains recognisable: fixed-priority, preemptive, statically limited and intended for small embedded systems. The new question is not whether all firmware is trusted equally, but where writable authority is deliberately shared.

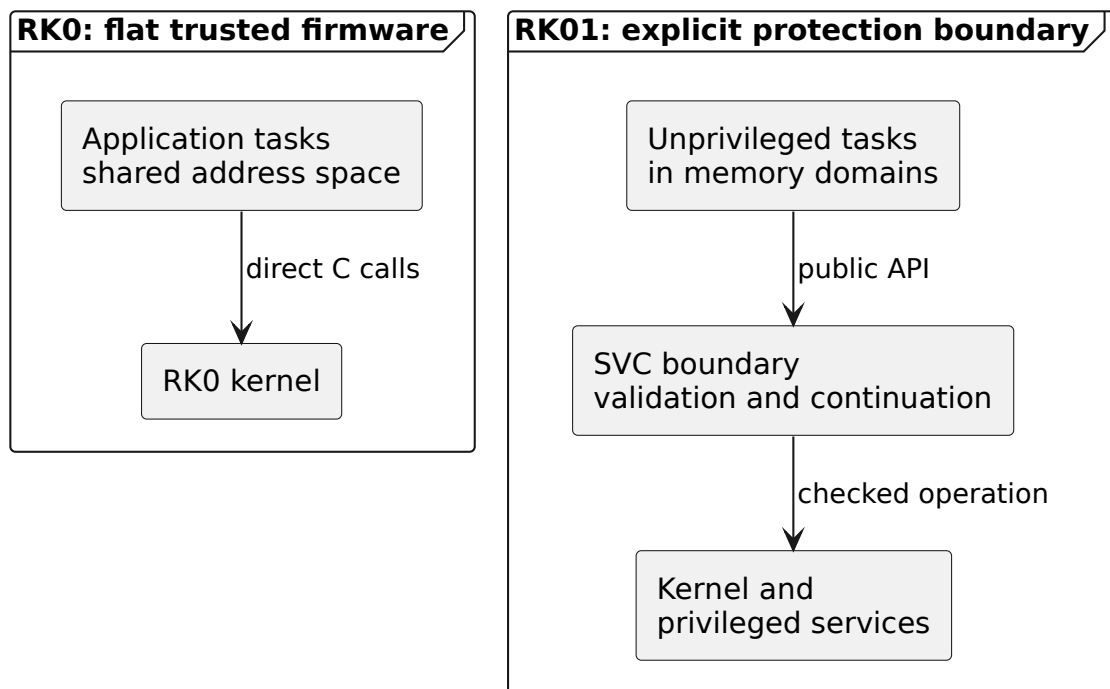


Figure 1: From flat trusted firmware to an explicit user/kernel protection boundary.

1.1 Threat model

RK01 assumes one intentionally built firmware image. The build, linker description, startup code, board port, kernel, exception paths and deliberately privileged services are trusted. Ordinary unprivileged task code may nevertheless be defective, memory-unsafe or compromised through an application-level error.

The protection objective is local containment:

- prevent an unprivileged task from writing kernel RAM;
- prevent it from writing another domain's persistent state;
- prevent it from writing another task's private stack;
- restrict its direct access to MMIO and privileged services;
- reject invalid handles and caller pointers at the syscall boundary;
- remove a faulted task from kernel protocols consistently.

RK01 does not claim protection against malicious privileged code, a defective exception handler, unrestricted DMA, debug or physical access, a dishonest build, or arbitrary code loading. Privileged code is not somehow shielded while it exercises universal authority. Reducing the amount of code that needs that authority reduces the trusted computing base; it does not abolish it.

2 System model and terminology

The term *domain* is intentionally narrower than *process* and different from a separately built module.

Term	Meaning in RK01
Task	The independently scheduled entity. It owns execution context, priority, state, timeout participation and a private stack aperture.
Domain	A named writable-authority boundary. One or more tasks may share its persistent RAM and the same explicit shared-region attachments.
Implicit App domain	The default domain used by ordinary application tasks created through the RK0-like task declaration and initialisation path.
Explicit domain	A BOOT-created domain with an explicitly declared MPU-compatible RAM window and one or more member tasks.
Isolated task	A one-task domain whose declared stack storage is also its private domain window. It is still an ordinary scheduled task, not a new scheduler class.
Privileged system task	A task deliberately running with privileged CONTROL state, such as Console, Logger, SysMon or PostProc. It remains inside the trusted computing base.
Source bundle	A C project-organisation convention: public header, private header, domain declaration, BOOT function and member-task sources. Its files compile to ordinary object files and participate in the same final firmware link.

Two tasks belong in one domain when they are intentionally allowed to modify the same persistent state. Domain membership is therefore a writable-fault boundary: a wild store into domain RAM by either member can damage state used by the other members. RK01 still protects their stacks from one another.

Creating more domains is not automatically better. A boundary is justified when fault containment, ownership or service structure is worth additional MPU, syscall and communication work.

3 Construction of the memory topology

3.1 One image and one final link

RK01 does not produce `kernel.elf`, `domain1.elf` and `domain2.elf`, nor does it load independent application images at run time. Domain source files may of course compile independently into `domain_a.o`, `domain_a_server.o` and so on, but those object files are inputs to one final link.

The important information is not the object filename. It is the section into which each writable declaration is emitted.

The ARMv7-M linker script classifies writable authority into the following principal areas:

Area	Current STM32F401RE placement	Purpose
FLASH	256 KiB at 0x08000000	Shared executable code and read-only data.
FS_FLASH	256 KiB at 0x08040000	Reserved storage used by RKFS; not part of the ordinary executable user window.
TASK_RAM	64 KiB at 0x20000000	Implicit App-domain RAM, explicit domain windows, private task stacks and shared apertures.
KERNEL_RAM	32 KiB at 0x20010000	TCBs, object pools, registries, kernel data, privileged stacks and the MSP.

The exact sizes are board-port choices, not universal RK01 constants. The classification is the architectural point.

Within TASK_RAM, the current ARMv7-M script reserves:

- a naturally aligned 16 KiB window for the implicit App domain;
- a generic collection for explicit `.rk_domain_ram*` windows;
- a distinct `.rk_task_stack*` collection for ordinary private stacks;
- task no-init storage;
- a fixed 1 KiB global shared aperture;
- optional explicit shared regions declared by the application.

Kernel `.data`, `.bss`, no-init data, object pools and privileged stacks go to KERNEL_RAM. Shared code remains in FLASH. This is why the linker is best described as a build-time classifier of writable authority.

3.2 Domain source bundles

A useful domain layout is:

File	Responsibility
<code>domain_a.h</code>	Public request, reply and export types. It should not expose private writable representation.
<code>domain_a_internal.h</code>	Typed private RAM layout, member-task declarations and implementation-only definitions.
<code>domain_a.c</code>	The domain window and descriptor, BOOT construction, public exports and composition of member tasks.

File	Responsibility
<code>domain_a_worker.c</code>	One or more member-task implementations using the private typed RAM view.

There is no automatic placement of a global merely because it appears in `domain_a_worker.c`. A casual writable global still follows the normal linker rules and will commonly land in the implicit App domain. RK01 therefore makes private persistent state explicit through a typed domain RAM declaration. The build's `DOMAIN_IMPL_SRCS` audit warns when a source-bundle implementation introduces unexpected `.data`, `.bss`, `.sdata`, `.sbss` or `COMMON` symbols.

That audit is a construction aid, not the MPU itself. The MPU enforces the final linked map; the audit helps the author ensure that the map expresses the intended ownership.

3.3 Sequential build and BOOT process

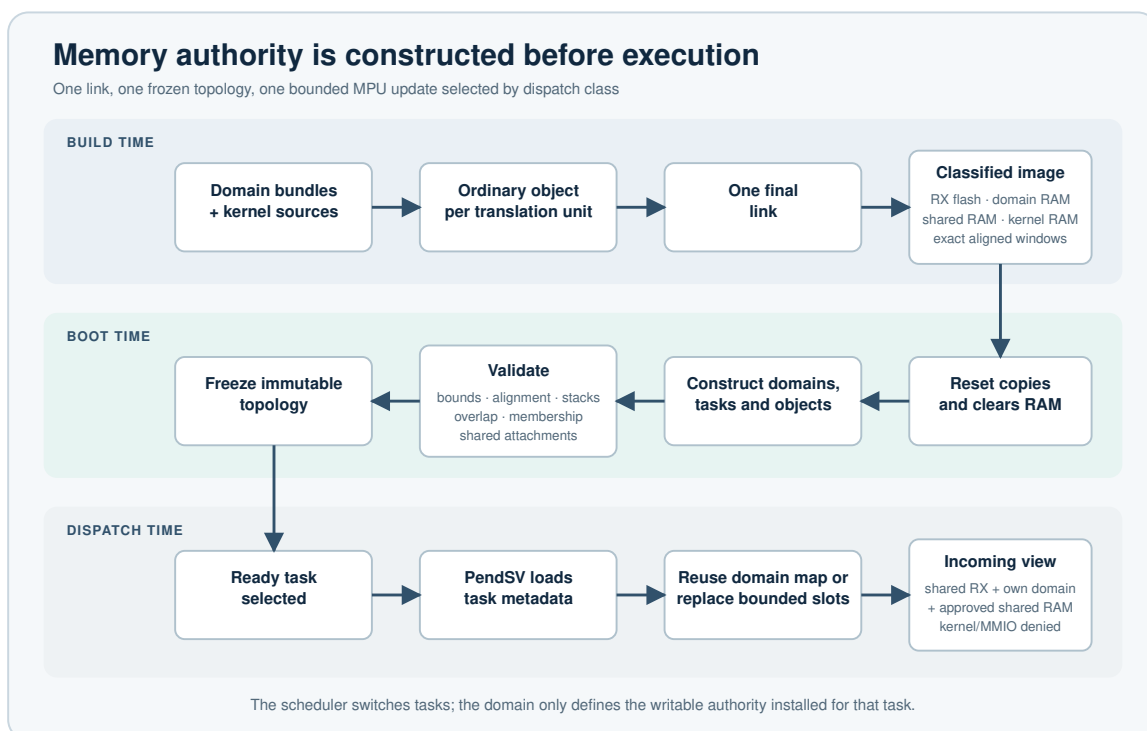


Figure 2: Construction of writable authority across build, BOOT and dispatch.

The sequence is important:

1. Declarations create intentionally named writable sections.
2. Each source file is compiled normally.
3. The final linker collects, aligns and bounds those sections and exports startup symbols.
4. Reset startup copies initialised data and clears the appropriate zero-initialised areas.
5. Privileged BOOT code initialises domains, allocates typed domain state, creates tasks and attaches explicit shared regions.
6. Before scheduling begins, the kernel validates region size and alignment, `TASK_RAM` containment, non-overlap, private-stack placement, domain membership and shared mappings.
7. The topology is frozen.

8. PendSV dispatches tasks using precomputed MPU descriptors.

The kernel does not discover or construct an address space dynamically inside every context switch. Dispatch consumes a topology that was already validated.

3.4 Why a prebuilt kernel object is not the domain model

RK01 could technically be distributed as a precompiled archive or `kernel.o`, provided its configuration-dependent ABI, pool geometry, feature set and required linker symbols were frozen. That is a packaging choice, not a protection feature.

The current model lets the application configure and compile the kernel because configuration changes code, object capacities and memory requirements. Compiling each domain to one relocatable object may help encapsulation or proprietary delivery, but it does not create domain placement automatically and does not improve MPU isolation. The final linker must still classify the sections and establish concrete addresses.

4 MPU views and context-switch classes

4.1 The effective task view

On the current ARMv7-M port, an ordinary task's view is composed from fixed MPU roles:

Region	Authority	Update rule
0	Shared user FLASH, executable and read-only	Installed once during MPU initialisation.
1	Active domain RAM	Replaced when dispatch crosses a domain boundary.
2	Active task's private stack	Replaced on every dispatch to a different task.
3	Global shared RAM	Part of the domain authority map; reloaded on a domain change.
4—7	Explicit shared regions attached to the active domain	Part of the domain authority map; reloaded on a domain change.

Privileged handlers and privileged system tasks use the default privileged system map through `PRIVDEFENA`. Unprivileged tasks see only the enabled user regions.

The private stack overlay is a deliberate difference between “these tasks share domain state” and “these tasks may overwrite one another's execution frames.” RK01 permits the first without permitting the second.

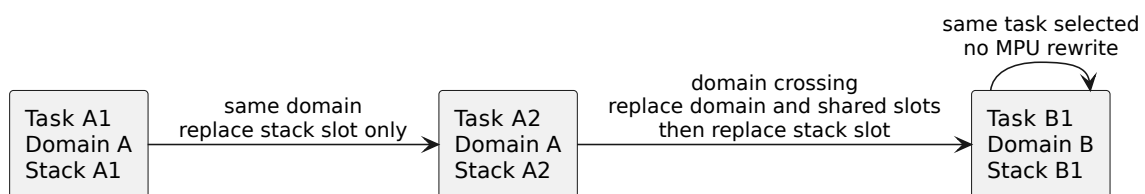


Figure 3: The three MPU-dispatch classes: no rewrite, stack-only update and full domain transition.

4.2 Immutable domain maps

The current dispatch path remembers both the active task and active domain. When the selected task is unchanged, it performs no MPU write. When the task changes but its domain is the active domain, it changes only the private-stack slot. When the domain changes, it reloads the domain and shared-authority slots before installing the selected stack.

Dispatch class	MPU work	Meaning
Same task selected again	None	The installed task view is already correct.
Different task, same domain	Replace private-stack region	Persistent state and shared attachments remain mapped; execution frames remain private.
Different domain	Replace domain RAM, global/explicit shared slots and private stack	Writable authority changes completely.

This is the practical reason for grouping cooperating tasks into domains. An intra-domain hand-off retains containment from unrelated domains while avoiding a complete authority-map reload. An inter-domain hand-off pays the larger cost because it changes the fault boundary.

5 Object identity, scope and lifecycle

5.1 Why a pointer is no longer a sufficient handle

In RK0, a handle can naturally be a pointer to an object because the caller already shares the kernel's trusted address space. Across an unprivileged boundary, an arbitrary caller-supplied address cannot be accepted as a mutex, semaphore, queue or TCB.

RK01 task-facing handles are opaque tokens. The 32-bit encoding contains:

- a fixed RK tag;
- an object-kind field;
- a 16-bit generation;
- an 8-bit registry slot.

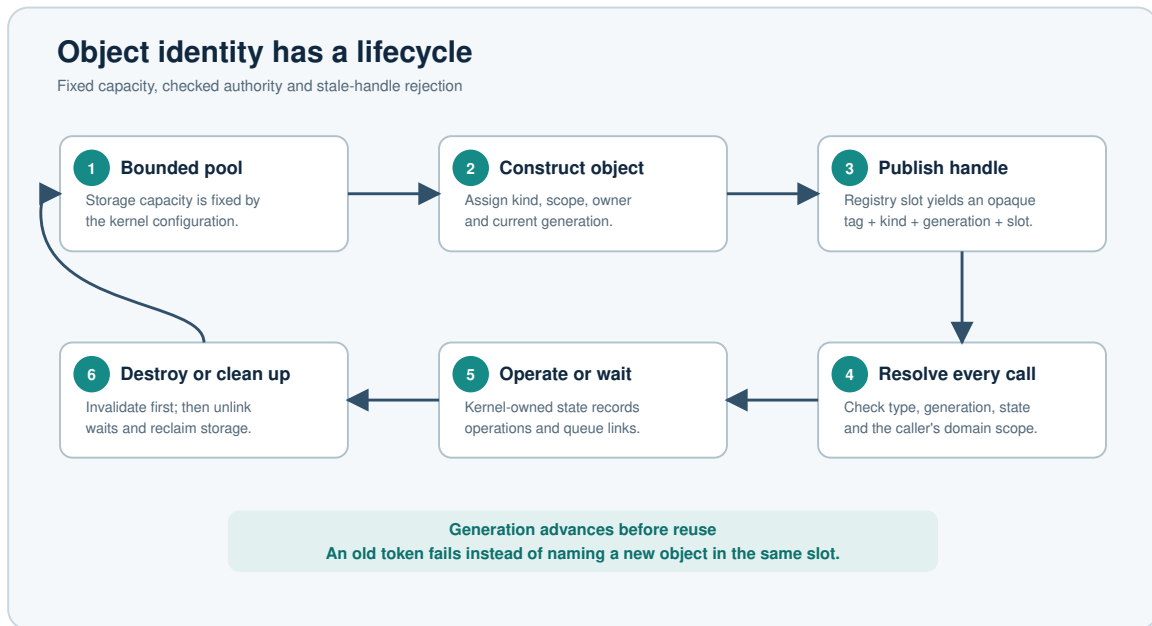


Figure 4: *Generation-checked object identity and bounded lifecycle.*

Decoding the fields is not sufficient to establish validity. Resolution also checks the configured family capacity, current generation, live registry entry, ownership by the correct fixed pool and the internal object type header.

Task handles use the same slot-and-generation principle through the live-task registry. The application never needs a TCB address.

5.2 Bounded lifecycle

Run-time creation is therefore not an open-ended heap. It is reservation from a build-time-bounded family pool. Creation time may be dynamic; maximum capacity is not.

5.3 Scope is separate from possession

Every scoped object records whether it is:

- `RK_SCOPE_DOMAIN_LOCAL`, with an owning domain; or
- `RK_SCOPE_KERNEL_GLOBAL`, intentionally usable across domains.

Holding the numerical token is not, by itself, authority. The kernel resolves identity and checks scope against the current caller domain. This is a substantial difference from protection schemes in which MPU placement guards raw loads and stores but an RTOS API will still operate on any valid object ID supplied through privileged handler mode.

Domain-local is the natural default for mutexes, semaphores, sleep queues and pointer-bearing services. Global scope is deliberate infrastructure for cross-domain queues, mailboxes or other system-wide endpoints.

5.4 Fault-aware cleanup

An object model is credible only if abnormal task exit cannot leave its protocols silently valid.

When an unprivileged task is retired after a fault, RK01 invalidates its task identity, removes wait and timeout links, clears syscall continuation state and releases message relationships. A mutex owned by that task is poisoned; waiters receive `RK_ERR_MUTEX_OWNER_FAULTED` rather than inheriting ownership of state whose invariants may already be broken. Reuse requires destroy/recreate or a wider recovery action.

The kernel can make its own queues and identities consistent. It cannot infer whether the application's data remains meaningful.

6 The system-call boundary

6.1 Two entry paths behind one public API

RK01 retains a conventional C API. The wrapper first determines its execution context:

- handler mode and privileged thread mode call the implementation directly where permitted;
- ordinary unprivileged thread mode enters through SVC.

This lets privileged BOOT and system services avoid a pointless trap while ensuring that ordinary tasks cannot bypass the boundary.

The SVC path derives the caller from the current TCB. Caller identity is never trusted as an argument. Before privileged code dereferences application-supplied data, the dispatcher validates:

- trap origin and syscall number;
- handle tag, kind, slot, generation and liveness;
- object scope against the caller's domain;
- required input ranges for readability;
- required or optional output ranges for writability;
- callback or function pointers as valid user code addresses;
- packed argument records, copied into kernel-local storage before nested pointers are interpreted.

6.2 Immediate and blocking calls

An immediate syscall completes in handler mode, writes the result into the caller's stacked R0, and returns.

A blocking syscall cannot simply preserve an arbitrary privileged C call chain on the MSP while unrelated tasks and nested syscalls execute. RK01 represents the continuation explicitly in the caller's TCB: saved exception frame, syscall number, original arguments, phase and wake result.

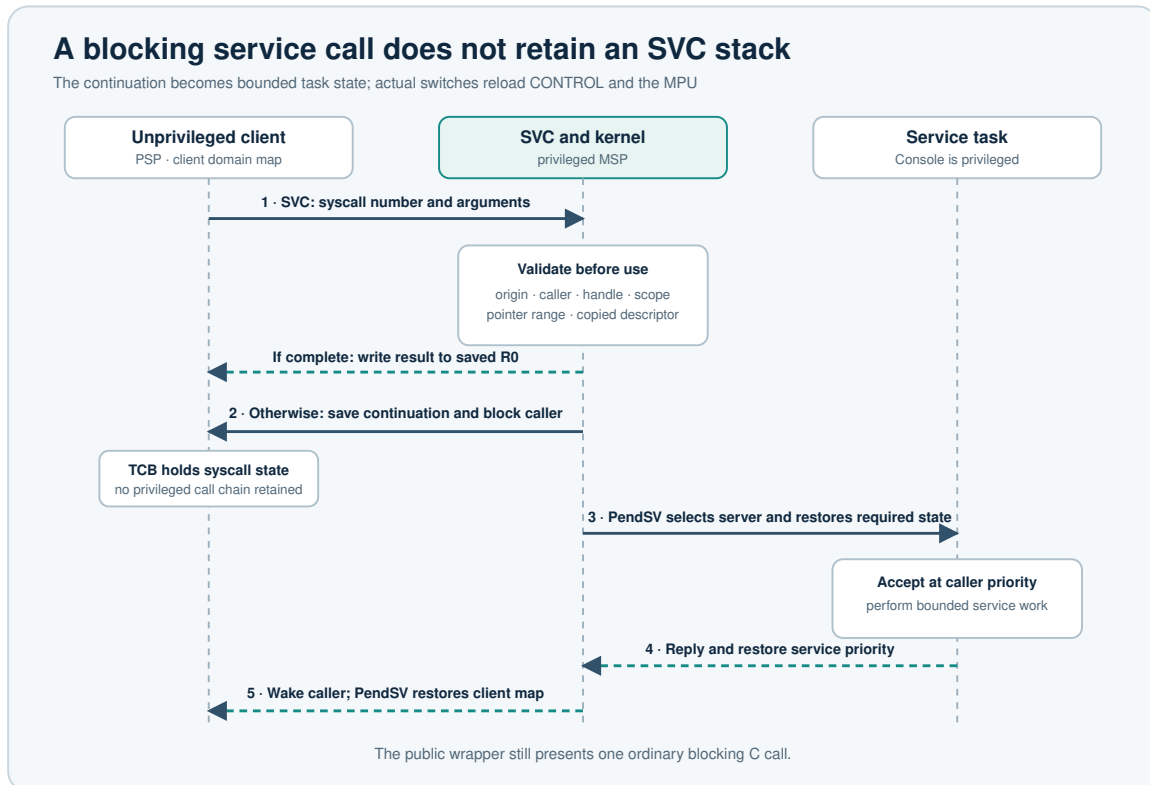


Figure 5: A blocking service call represented by bounded task state rather than a retained privileged stack.

The public function still appears to block once. Internally it may cross SVC more than once, with the continuation represented as bounded task state rather than a suspended privileged stack.

Long non-blocking kernel work may use the same restart principle at explicit checkpoints. This allows a pending higher-priority task to run without exposing an unbounded privileged continuation.

6.3 Timing consequence

A syscall is a protocol, not merely the SVC instruction. Its response-time contribution may include exception entry and exit, validation, object lookup, wait-queue operations, PendSV, an MPU update, later wakeup and continuation. A client/server service adds at least the server dispatch, service execution, reply and client redispach.

Those costs are finite and classifiable, but they are not equal to RK0's direct function call.

7 Communication follows memory authority

RK01 does not impose message passing inside every domain. Tasks that intentionally share domain RAM can use ordinary loads and stores and coordinate with familiar services. Forcing such traffic through a server would add states and context switches without creating a new containment boundary.

Across domains, a pointer is meaningful only when both views map the referenced storage. RK01 therefore distinguishes communication contracts explicitly.

Mechanism	Natural use	Boundary rule
Direct shared state	Closely cooperating tasks in one domain	Both tasks already map the same domain RAM; synchronisation is still required.
Domain-local semaphore, mutex or sleep queue	Coordination around domain-owned invariants	The object's owner scope is checked independently of handle possession.
Task event	Cross-domain notification without a payload pointer	Targets a task identity; no writable object is transferred.
Asynchronous direct message	Ownership transfer of an application-owned message buffer	By-reference use is same-domain because the receiver must retain access to the buffer.
Asynchronous copied message	Queued payload across non-shared domains	Payload moves through bounded kernel-owned message storage.
Synchronous send/receive	Blocking one-way rendezvous	The receiver copies the payload; there is no reply and no receiver priority substitution.
Synchronous call/accept/reply	Request/reply service	Copied request/reply with one active caller per server and caller-priority substitution while queued or active.
Global queue or mailbox	Named infrastructure shared by several domains	The object must be explicitly global; payload storage must still satisfy its copy/reference contract.
Explicit shared memory	Deliberately shared high-rate or stateful data	The segment is attached to selected domains before topology finalisation and consumes MPU capacity.
MRM	Latest-value publication using pointer leases	Domain-local under the current API because a lease is a pointer. Across domains, use a copied payload or an explicit shared-memory snapshot instead.

The policy is not “messages are good and shared memory is bad.” The policy is that the communication primitive must match the writable authority actually installed by the MPU.

7.1 Priority-aware service calls

Call/reply is intended for a server that owns state or hardware on behalf of clients. While calls are queued or active, the server's effective priority reflects its most urgent caller. This limits priority inversion caused by a low nominal server priority. Timeout, reply and cleanup remove the relationship.

Priority substitution does not make the service free. It makes the scheduling dependency explicit enough to analyse.

8 UART Console as a complete service example

The Console path demonstrates why a client/server structure becomes natural once ordinary tasks no longer own MMIO authority.

8.1 Topology

- Ordinary clients run unprivileged in one or more domains.
- USART2 MMIO remains accessible to privileged code.
- A privileged Console task owns foreground transmit and receive processing.
- A small ISR transfers received bytes into a fixed ring and signals the Console task.
- Transmit uses synchronous call/accept/reply.

8.2 Transmit path

An unprivileged task calls `kConsoleWrite()` with a buffer and byte count. A zero-length request succeeds immediately. The front end rejects a null buffer and any request beyond `RK_CONF_CONSOLE_WRITE_MAX_BYTES`, which is 65 bytes in the default configuration.

The caller builds a bounded request descriptor and reply area on its private stack. The syscall copies and validates the protocol metadata, then enters the generic call/reply machinery. The client blocks while the privileged Console task accepts the request.

The descriptor contains a pointer to the original payload. Before using it, the Console task resolves the original caller and validates that the requested range is readable in that caller's MPU contract. The caller remains blocked until reply, so its private stack and syscall continuation remain stable during this bounded loan. The current example transmits by polling the UART-ready condition; a future interrupt- or DMA-driven implementation would change the device-side execution without changing the protection protocol.

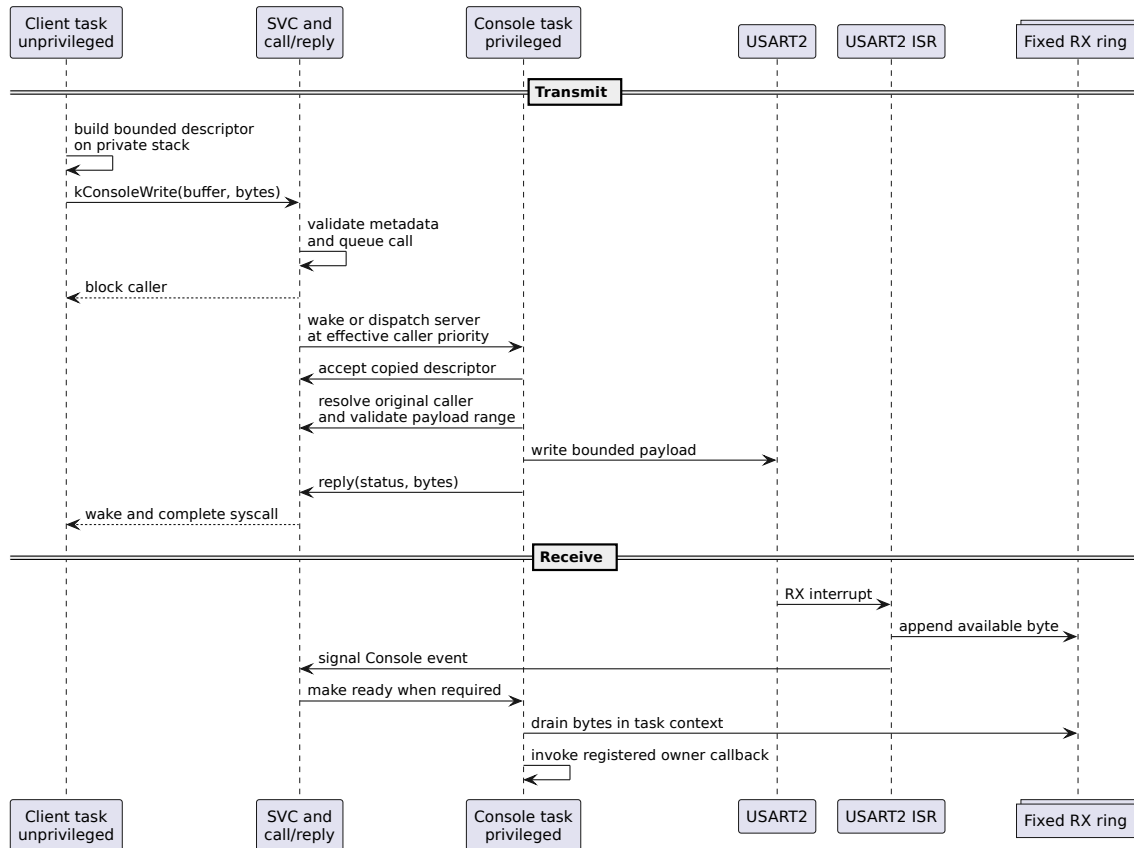


Figure 6: UART Console transmit and receive paths across the protection boundary.

8.3 What the example proves

The example does not prove that all drivers must be server tasks. It demonstrates a coherent boundary:

- one privileged endpoint owns the peripheral;
- clients use a bounded checked interface;
- call/reply represents blocking service ownership;
- priority substitution connects client urgency to server scheduling;
- the ISR remains small;
- application parsing and policy can remain outside the interrupt path.

It also exposes the trusted-base issue honestly. The Console task and any callback it invokes while privileged are trusted. Moving parsing or protocol policy into an unprivileged domain can reduce that trusted surface; merely naming privileged code a task cannot.

9 Fault containment and application recovery

9.1 What RK01 does on an MPU violation

For a valid exception frame originating from an ordinary unprivileged task, MemManage records the task ID, PC, LR, status registers, fault address when available, PSP and CONTROL state. It clears the processor fault status, marks the running task `FAULT_PENDING`, schedules away and queues cleanup for PostProc.

PostProc later invalidates public task identity and unlinks kernel relationships. A fault in handler mode, privileged thread mode or an unusable exception context is fatal because the kernel cannot safely treat it as an isolated user-task failure.

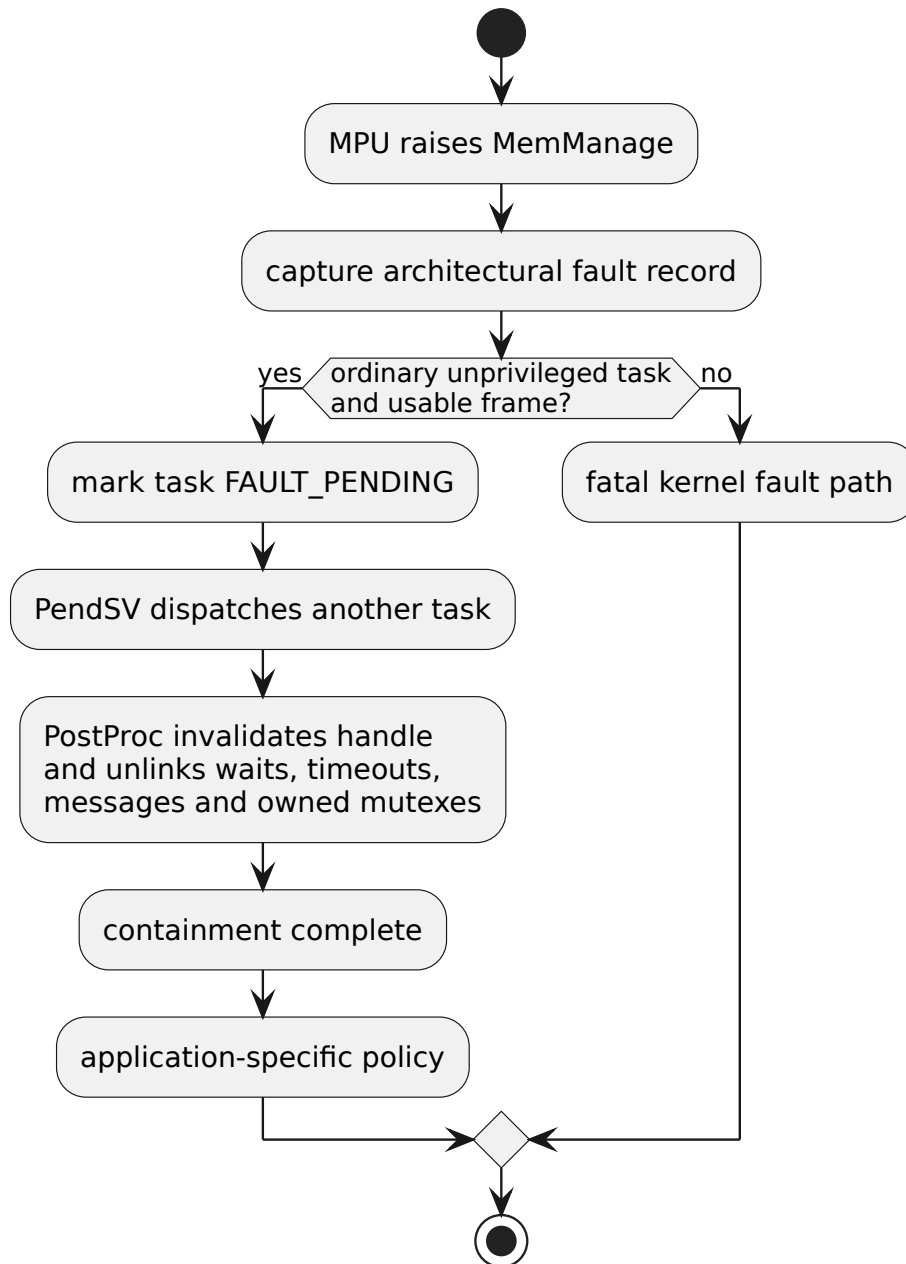


Figure 7: Containment path for an eligible unprivileged task fault.

9.2 Containment is not recovery

Killing the offending task prevents further execution. It does not reconstruct an application invariant that may have been damaged before the illegal access.

Recovery therefore depends on what the product controls. Possible policies include:

- command outputs to a safe state and continue in degraded mode;
- fail over to a pre-created redundant domain;

- reinitialise a peripheral and its owning service state;
- abandon the affected function until maintenance;
- reset the controller when shared invariants cannot be established safely.

9.3 Example: failover rather than blind restart

Consider a controller with `PrimarySensor` and `StandbySensor` domains. Each publishes copied samples to a separate `Control` domain. No mutable sensor state is shared directly with `Control`.

1. A task in `PrimarySensor` violates its MPU map.
2. RK01 records and contains the task, then `PostProc` cleans its kernel relationships.
3. A bounded privileged fault-report handoff informs a product supervisor. This notification path is application integration work; the V0.1.0 diagnostic record alone is not a complete supervisor protocol.
4. The supervisor commands the actuator controller to its defined transient-safe value.
5. It marks the primary sample stream invalid using a generation or epoch field and selects the already-running standby stream.
6. Remaining tasks in the primary domain are quiesced by application policy because their shared domain state can no longer be trusted.
7. The system either remains degraded or schedules a controlled reset at an acceptable process state.



Figure 8: Application-dependent failover after kernel-level containment.

This example is feasible because it does not pretend that zeroing one stack repairs shared domain state. A transparent in-place restart would require more kernel and application machinery: domain-level lifecycle, deterministic RAM reinitialisation, invalidation and reconstruction of domain-local objects, task recreation rules and a bounded supervisor notification path. Those are legitimate future features, not properties supplied automatically by the MPU.

10 Building an operating system around RK01

RK01 supplies kernel mechanisms; it does not prescribe the complete operating-system structure built from them. The product defines the topology during BOOT: which tasks exist, which tasks share a domain, which services execute privileged, which kernel objects are domain-local or global, and which shared-memory regions are deliberately attached.

The kernel validates and freezes that topology before dispatch. It does not infer architecture from directories, source filenames or separately linked component images. A domain source bundle is an organisational convention that helps the application author express the topology; the final result remains one firmware image.

This leaves several legitimate ways to construct a system around RK01.

10.1 Three ways to compose a system around RK01

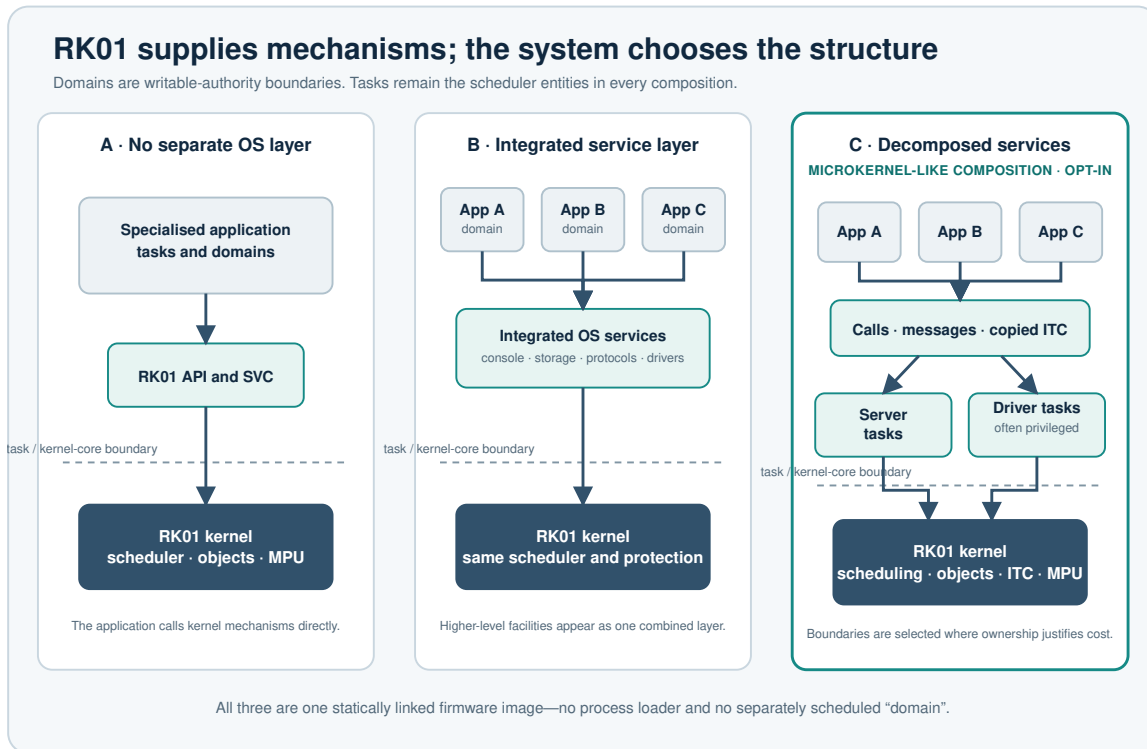


Figure 9: Three valid ways to compose a system around RK01.

10.1.1 A specialised application with no separate OS layer

The smallest arrangement resembles traditional RK0 usage. Application tasks call the RK01 API directly, and no separately identifiable user-side operating-system layer exists.

The calls made by ordinary tasks still cross SVC, and the tasks still execute within MPU-enforced domains. “No OS layer” therefore does not mean “no protection.” It means that the application uses kernel mechanisms directly rather than accessing them through service tasks or a product framework.

This arrangement minimises architectural indirection. A semaphore operation, sleep request or message transfer does not need to be translated into an application-defined service protocol. Where the kernel operation completes immediately, crossing the user/kernel boundary does not imply dispatching another task.

Such a structure is appropriate for specialised firmware in which the application logic already expresses the complete system and an additional service layer would merely reproduce the kernel API.

10.1.2 An integrated user-side system layer

A larger product may place protocols, storage policy, diagnostics and common application facilities behind an integrated framework above RK01.

This framework may occupy one domain or several domains. To its clients it can appear as one cohesive OS-facing API even though its implementation consists of independently scheduled tasks with different writable-memory authorities.

This structure remains comparatively monolithic at the application level:

- clients depend on one integrated framework;
- some state is intentionally shared among framework tasks;

- selected services may remain ordinary library calls;
- kernel entry still occurs through the RK01 API;
- hardware access remains privileged or passes through a protected endpoint.

The advantage is that closely related facilities can cooperate without converting every interaction into copied IPC. The cost is a wider containment boundary wherever those facilities share a domain.

10.1.3 A service-oriented system

At the other end of the range, selected facilities may be implemented as independently scheduled servers. Application domains send requests instead of directly owning hardware, persistent service state or complex shared subsystems.

A service may use a domain containing exactly one task. This is natural for a UART server, watchdog manager, storage server or other component whose state and execution belong together. It is still an ordinary RK01 domain and an ordinary scheduled task; there is no special “server” scheduler entity.

A more elaborate subsystem may place several cooperating tasks in one domain. They then share the subsystem’s persistent state and domain-local objects while retaining private task stacks. Context switches inside that subsystem retain the installed domain map and replace only the private-stack window.

A real system can mix both forms:

- one-task domains for independently isolated services;
- multi-task domains for tightly coupled subsystems;
- ordinary application domains that call the kernel directly;
- privileged endpoint tasks for MMIO and exception-facing work;
- explicit shared regions where copying would be inappropriate.

Consequently, RK01 does not impose either “one domain per task” or “many tasks per domain.” Domain granularity is part of the product architecture.

Composition	Typical interaction	Primary benefit	Primary cost
Direct application	Application task invokes a kernel service through SVC	Shortest protected mechanism path	Application structure remains coupled directly to the kernel API
Integrated framework	Application calls a shared user-side abstraction	Cohesive product API and inexpensive cooperation inside selected domains	Larger shared-state and failure boundaries
Service-oriented system	Client calls an independently scheduled server	Explicit ownership, isolation and replaceable service policy	Additional validation, copying and task dispatches
Split service and endpoint	Unprivileged service calls a narrow privileged hardware endpoint	Most service logic remains outside the privileged trusted base	Another protocol boundary and potentially more context switches

11 Why the boundaries encourage a microkernel-like composition

A client/server architecture can be constructed on almost any multitasking kernel. What changes in RK01 is that the protection model makes such an architecture a natural consequence of ordinary design decisions.

An unprivileged task cannot simply dereference kernel data or device registers. Cross-domain pointers are not automatically meaningful. Kernel entry is already explicit. A device needs a clearly identified owner. Once those facts are accepted, representing a driver or system facility as a service task becomes a direct way to express the boundary.

Several RK01 mechanisms reinforce that structure:

- SVC provides a checked transition into privileged mechanisms;
- domain membership defines writable authority;
- domain-local object scope limits accidental cross-component use;
- copied messages cross domains without granting memory access;
- synchronous call/reply expresses a service invocation;
- priority substitution lets the server execute with the effective urgency of its caller;
- small interrupt handlers can defer substantial work to scheduled service tasks;
- task-fault cleanup prevents a failed client from remaining linked into kernel wait and ownership structures.

These properties allow an OS built around RK01 to adopt microkernel-like organisation selectively. They do not force all software into servers.

11.1 Placement is a system-design decision

A facility can be placed at several levels:

Placement	Authority	Execution cost	Design consequence
Kernel mechanism	Universal privileged authority	No service-task dispatch	Smallest path, but permanently enlarges the kernel's trusted code
Privileged service task	Privileged access, including required MMIO	Client/server dispatch and reply	Separates scheduling and ownership, but the service remains in the trusted base
Unprivileged service domain	Only its declared domain and shared authorities	Cross-domain call, validation and possible copying	Removes service policy from the privileged trusted base
Unprivileged service plus privileged endpoint	Service logic is restricted; the endpoint owns MMIO	At least one additional protected interaction	Narrows privileged driver code at a measurable timing cost
Direct application task	Its domain and permitted kernel API	SVC without an intermediate server	Appropriate when a separate service would add no useful ownership or containment boundary

A strict microkernel would attempt to move most drivers and system services into unprivileged address spaces. RK01 does not require this. A privileged service task is allowed when the

additional transition through a separate unprivileged driver domain would cost more than the containment is worth.

Conversely, a complex protocol parser does not need to remain privileged merely because it ultimately controls a device. It can execute in an unprivileged domain and communicate with a much smaller privileged endpoint that performs bounded register operations.

The boundary can therefore be placed according to the fault model and timing budget rather than according to a categorical definition of “microkernel.”

11.2 The UART service as a composition example

The current Console task illustrates the middle point of this range.

An unprivileged client calls `kConsoleWrite()` with a bounded request. SVC validates the request metadata and transfers it through synchronous call/reply. The privileged Console task accepts the operation, validates the original caller’s payload range, accesses USART2 and replies. The caller remains blocked until completion.

On receive, the interrupt handler performs only bounded device-facing work: it obtains the available byte, appends it to a fixed ring and signals the Console task. The scheduled task drains the ring and performs the higher-level processing.

This structure already has microkernel-like properties:

- the client does not own the UART;
- device access has one explicit owner;
- requests cross a checked service boundary;
- the interrupt handler remains small;
- the driver executes as a schedulable task;
- the server can inherit the effective urgency of the caller.

Nevertheless, the Console task is privileged. It can access privileged memory and therefore remains part of the trusted computing base. Calling this arrangement “microkernel-like” describes its client/server organisation; it does not claim that the driver has been removed from the privileged system.

A more strongly isolated design could split the service:

1. An unprivileged Console domain would implement framing, buffering and application-facing policy.
2. A small privileged USART endpoint would perform only bounded register access and ISR coordination.
3. The Console domain would call that endpoint through a narrow protocol.

That decomposition would reduce the amount of privileged driver code, but it would add another service transition, more validation and possibly another pair of context switches. It is justified only if isolating the higher-level driver logic is worth that cost.

The opposite design is also possible. A UART syscall could perform the complete bounded transmission inside the kernel without dispatching a Console task. This would reduce task-switch overhead, but it would place the operation and any polling delay directly in a privileged kernel path.

RK01 permits all three arrangements. The current privileged Console task represents a deliberate compromise between a kernel-resident driver and a fully split unprivileged driver stack.

11.3 The real-time cost remains visible

A microkernel-like structure is not free. A direct syscall may require only SVC entry, validation, the kernel operation and exception return. A synchronous server invocation additionally requires request handling, dispatch to the server, server execution, reply processing and redispach to the client.

If another unprivileged service and privileged endpoint are inserted, their crossings must also be included in the response-time path.

Priority substitution limits service-induced priority inversion, but it does not eliminate:

- context-switch execution;
- MPU reprogramming on inter-domain dispatch;
- payload validation;
- request and reply copying;
- wait-queue manipulation;
- cache or bus effects on applicable targets;
- execution time inside the service itself.

Singleton service domains maximise isolation, but every switch between distinct services is an inter-domain switch. Grouping cooperating tasks into one domain permits shared state and cheaper intra-domain dispatch, but makes the entire group one writable-state trust boundary.

Neither arrangement is universally correct. RK01 keeps both bounded and makes the distinction visible so it can be included in response-time analysis.

11.4 Containment is supplied by the kernel; recovery belongs to the OS

The separation between kernel mechanism and OS composition becomes most important after a fault.

RK01 can identify an eligible unprivileged task fault, remove that task from execution and clean its kernel relationships. It cannot determine what the failed task meant to the product.

If the task was a replaceable worker, the surrounding system may continue without it. If it was the only server for a critical facility, its clients need a defined failure response. If several tasks shared its domain state, the entire domain may need to be quiesced even though only one task faulted. A redundant service may be selected, or the product may enter a transient-safe state and perform a controlled reset.

These are operating-system and application policies built around RK01:

- supervision;
- service-availability tracking;
- failover;
- domain quiescence;
- state reconstruction;
- degraded operation;
- controlled restart or reset.

The MPU makes containment possible. It does not make recovery generic.

11.5 Architectural position

RK01 is therefore neither a compulsory microkernel nor merely a flat kernel with memory checks.

At one extreme, a specialised application can call kernel services directly and introduce no user-side OS layer. At the other, an OS can place each substantial service in its own unprivileged domain and retain only narrow privileged endpoints. Between those extremes, the product may group cooperating tasks, retain selected privileged servers and use direct kernel services where additional separation would provide no useful containment.

RK01 itself remains one statically linked, fixed-priority real-time kernel containing substantial scheduling, object, ITC, logging and diagnostic machinery. What it provides is the option to construct a microkernel-like system around that core without imposing that organisation on every application.

The useful claim is therefore not that RK01 is a strict microkernel. It is that:

RK01 makes service-oriented, microkernel-like composition a natural and explicit option, while preserving direct bounded mechanisms where additional isolation would be an unjustified real-time cost.

12 Real-time properties and their price

12.1 What remains bounded

RK01 retains:

- fixed-priority preemptive scheduling;
- finite priority and wait structures;
- build-time-limited task and object pools;
- fixed MPU capacity;
- bounded syscall argument records;
- bounded copied-message storage;
- a topology frozen before the first dispatch;
- finite same-domain and inter-domain MPU update paths.

Run-time creation does not imply unbounded resource growth. The maximum number of live tasks and objects remains a build decision.

This does not mean all operations have the same cost, nor that a WCET argument is automatic. It means the design avoids mechanisms whose state space or work grows without a configured bound.

12.2 Response-time classes

A realistic analysis must distinguish at least:

Class	Components
Privileged direct call	Normal call overhead plus kernel operation.
Immediate syscall	SVC entry/exit, caller derivation, handle and pointer validation, kernel operation.
Blocking syscall	Immediate path plus wait insertion, PendSV, wakeup and one or more continuation entries.

Class	Components
Same-domain dispatch	Register context switch plus private-stack MPU update.
Inter-domain dispatch	Register context switch plus domain/shared-map update and private-stack update.
Call/reply service	Client syscall, request validation/copy, server dispatch and execution, reply, client redispach and continuation.

The relevant service equation is therefore structural rather than a single universal context-switch number:

Structural service-time decomposition

```

T_service = T_svc_entry
           + T_validation
           + T_request_protocol
           + T_dispatch_client_to_server
           + C_server
           + T_reply_protocol
           + T_dispatch_server_to_client
           + T_syscall_continuation

```

Each term can be bounded for a target and configuration. Some terms disappear for privileged direct calls or same-domain operations.

12.3 Preemption-chain measurement

The available ThreadX-style preemptive scheduling test uses five counter tasks arranged by priority [3]. The lowest-priority task drives the chain; each intermediate task resumes the next higher-priority task and suspends itself; the highest task increments and suspends. A reporting task samples cumulative counters every 30 seconds.

The reported build conditions were:

- STM32F401RE Cortex-M4F at 80 MHz;
- -O2 optimisation;
- NDEBUB enabled;
- FPU disabled;
- stack-overflow checking disabled;
- debug symbols retained.

After five 30-second intervals, the supplied measurements were:

Kernel	Total counter increments	Maximum counter spread	Test errors
RK0	26,766,655	0	0
RK01	7,124,887	1	0

RK01 therefore achieved approximately 26.62% of RK0's whole-chain throughput; conversely, RK0 completed approximately 3.76 times as many chain increments in the same interval.

This is not evidence that an isolated RK01 context switch is universally 3.76 times slower. The chain includes event syscalls, exception entry and exit, handle and pointer checks, event-state

transitions, blocking, wakeup, PendSV and the private-stack MPU update. All counter tasks use the implicit App domain, so the result does not include full inter-domain authority-map replacement.

The one-count RK01 spread also is not a numerical timing-jitter measurement. It is consistent with the reporting task interrupting the chain at one of several adjacent states. Both kernels satisfy the test's stated plus-or-minus-one counter criterion.

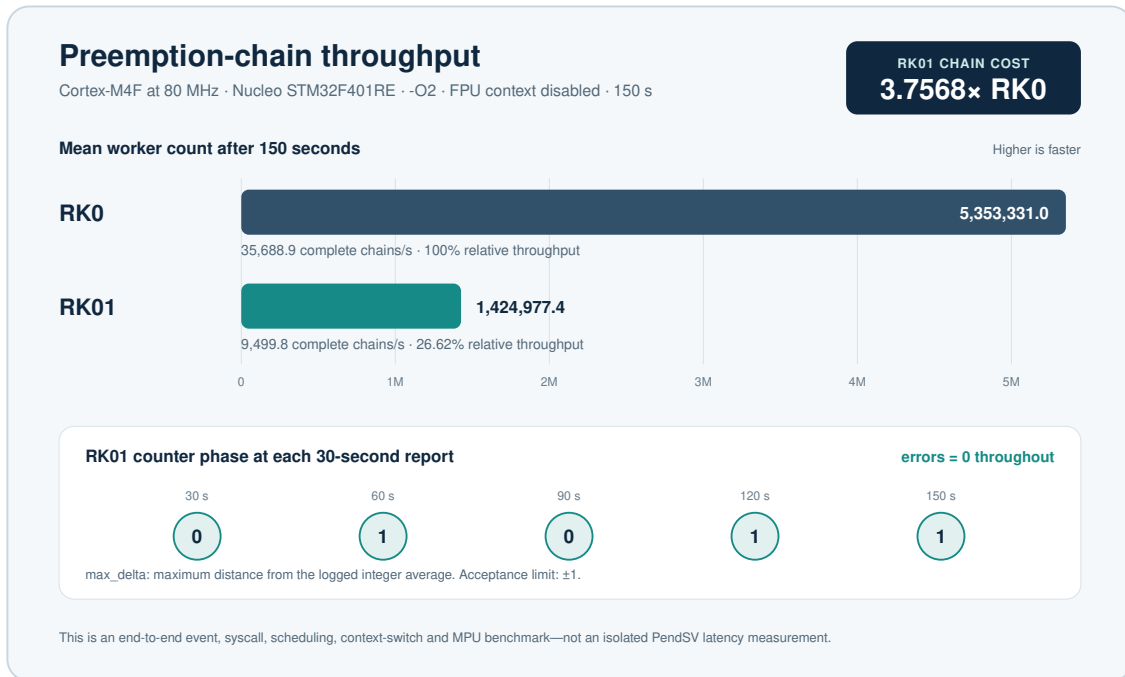


Figure 10: Whole-chain throughput in the supplied five-round preemption benchmark.

RK01's dedicated context-switch profile can build same-domain and inter-domain pairs separately. A defensible timing characterisation should report raw cycle counts and distributions for each path, calibrate instrumentation overhead on the target, and document interrupt, FPU, cache and debug conditions. Minimum, maximum and relevant percentiles are more informative than one average.

12.4 Hardware constraints versus real-time policy

Constraint	Hardware-derived	RK01 policy	Rationale
Finite MPU region count	Yes	RK01 assigns fixed roles to available slots	The processor cannot map an arbitrary number of independent windows.
Power-of-two size and natural alignment on ARMv7-M	Yes	Validated during BOOT	Invalid geometry cannot be repaired safely during dispatch and may waste RAM.
Separate task-stack aperture	No	Deliberate	Preserves stack isolation among tasks that share persistent domain state.
One linked code image	No	Deliberate	Avoids a loader, relocation, per-image validation and additional run-time states.

Constraint	Hardware-derived	RK01 policy	Rationale
Fixed global shared window size	Not inherently	Current port policy	Makes a scarce shared authority explicit and keeps its cost predictable.
Finite explicit shared attachments	Partly	Configured within the remaining MPU slots	Every attachment consumes map capacity and inter-domain dispatch work.
Topology frozen before dispatch	No	Deliberate	Moves validation and overlap analysis out of time-critical run-time paths.
No arbitrary process creation or remapping	No	Deliberate non-goal	General mapping and loading would introduce allocation, validation and timing variability that RK01 does not require.
Privileged code can access the default map	Architecture and configuration	Accepted trust model	The MPU contains unprivileged tasks; it does not isolate privileged components from one another.

Some restrictions exist because the small Cortex-M MPU leaves no alternative. Others would remain desirable even on more capable hardware because the omitted generality would enlarge the state space and response-time envelope. “Hardware could do it” is not, by itself, a reason for a small real-time kernel to expose it.

13 Design consequences and non-goals

13.1 The trusted computing base is reduced, not minimal

The kernel, exception paths, board support, privileged service tasks and callbacks executed by them retain privileged authority. A defect there can defeat containment. RK01’s improvement over RK0 is that ordinary application domains need not share that authority.

13.2 Shared code is intentional

All domains execute from the same linked FLASH image. Writable data, stacks, kernel RAM and shared apertures are protected. RK01 does not provide independent code loading, module-specific signing or separate link identities.

13.3 Domain boundaries do not replace synchronisation

Tasks inside a domain can race on their shared persistent state. MPU authority answers who may access memory; it does not serialize correct access.

13.4 Static topology is both feature and restriction

Freezing the map supports overlap validation, bounded dispatch and composition analysis. It rules out general run-time remapping and process-like construction.

13.5 Containment needs product policy

Fault retirement protects the rest of the system from continued execution. Whether safe operation can continue depends on the controlled process, redundancy, shared-state design and failure assumptions. Reset is sometimes the correct recovery policy.

13.6 “Microkernel-like” is an architectural choice

RK01 makes client/server services natural but does not require every facility to be a server and does not claim that a privileged server is outside the trusted base. The operating system built around RK01 can choose where the extra transitions buy a meaningful boundary.

14 Conclusion

RK01 is the RK0 real-time foundation with one additional boundary made explicit.

The domain is the central new abstraction, but it is deliberately modest: a frozen writable-authority map shared by tasks, not a process, loader or scheduler entity. The linker classifies storage; BOOT constructs and validates the topology; PendSV installs either a private stack overlay or a complete domain view. Opaque generation-checked handles prevent application tokens from becoming kernel addresses. SVC turns public API calls into checked protocols, with explicit task continuations for blocking work. ITC then follows the actual memory relationship: direct sharing inside a domain, copies or deliberate shared apertures across domains.

These mechanisms cost RAM, MPU slots, validation work and context-switch time. The available benchmark makes that price visible but does not reduce it to a misleading universal multiplier. The appropriate claim is narrower and stronger: RK01 keeps finite, classifiable mechanisms while adding fault-containment boundaries that RK0 intentionally does not provide.

There is no reason to merge both designs into a compromise. RK0 should remain flat, direct and transparent. RK01 should make its boundaries useful, bounded and honest about their bill.

References

- [1] A. Giacomelli, *RK0 source repository*, 2026. Available: <https://github.com/antoniogiacomelli/RK0>.
- [2] A. Giacomelli, *RK01 source repository*, development source, 2026. Available: <https://github.com/antoniogiacomelli/RK01>.
- [3] Eclipse ThreadX, *Thread-Metric benchmark: preemptive scheduling test*, 2026. Available: https://github.com/eclipse-threadx/threadx/blob/master/utility/benchmarks/thread_metric/thread_metric_readme.txt.